PROJETO

Estrutura de Dados

INTEGRANTES DO PROJETO

Nicollas Mota
Cauã Aono
Lucas Pereira
Arthur Carvalho

São Paulo

2026

Mini-Relatório: Complexidade e Estruturas de Dados

Projeto: Plataforma Marketplace B2B (Kakuab Market)

Objetivo: Demonstrar a aplicação de conceitos de estruturas de dados, bem como a análise de complexidade dos algoritmos de busca e ordenação nas camadas de Banco de Dados, Back- end e Front-end da aplicação.

1. Banco de Dados: Estrutura e Persistência

A arquitetura relacional (SQLite) fornece a base estrutural para as otimizações do sistema, garantindo a integridade do ecossistema B2B.

- **Validação em *O(1)*:** A tipagem estrita e o uso de *CHECK Constraints* (ex: limitando o tipo de usuário ou nota de avaliação) validam as inserções em tempo constante, rejeitando anomalias sem sobrecarregar a API.
- **Índices B-Tree e Buscas *O(log n)*:** A definição de Chaves Primárias (`PRIMARY KEY`) gera índices agrupados baseados em árvores balanceadas. Isso garante que buscas unitárias não realizem varreduras lineares.
- **Grafos Relacionais:** A utilização de Chaves Estrangeiras e restrições `UNIQUE` nas tabelas associativas (como avaliações) previne duplicidade e estrutura os dados de forma normalizada.

Exemplo SQL Aplicado: Organização Relacional

```sql
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    senha TEXT NOT NULL,
    tipo TEXT CHECK(tipo IN ('fornecedor', 'comprador', 'admin')) NOT NULL,
    status TEXT DEFAULT 'ativo',
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```sql
CREATE TABLE IF NOT EXISTS favoritos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    comprador_id INTEGER NOT NULL,
    anuncio_id INTEGER NOT NULL,
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (comprador_id) REFERENCES users(id),
    FOREIGN KEY (anuncio_id) REFERENCES anuncios(id),

    UNIQUE(comprador_id, anuncio_id)
);
```

Explicação Visual: Índices B-Tree

A chave primária (ID) organiza os dados em nós hierárquicos. Em vez de varrer milhares de registros de forma linear (*O(n)*), o motor de banco de dados desce pela árvore balanceada, dividindo o problema e encontrando o anúncio específico em poucas etapas operacionais, o que resulta em uma complexidade logarítmica *O(log n).*

2. Back-end: Lógica, Filtros e Ordenação

O Back-end (Node.js) atua como orquestrador, manipulando Objetos Literais (Hash Tables) em tempo constante *O(1)* e delegando o trabalho pesado de busca e ordenação para o banco de dados.

- **Ordenação Delegada** *O(n log n)***:** Ao invés de trazer Arrays desordenados para a memória

  RAM e rodar algoritmos nativos do JS, o sistema aplica a cláusula `ORDER BY` no SQL. O SQLite utiliza variações de algoritmos eficientes de ordenação, entregando os dados já classificados.

- **Otimização Relacional (Prevenção N+1):** O uso de `INNER JOIN` nas consultas funde a busca de tabelas distintas em uma única operação em disco, reduzindo drasticamente a complexidade de tempo em listagens de anúncios ativos.

Trecho Real de Código: Busca Avançada Otimizada

```
function listarAnunciosPublicos(callback) {
  const sql = `
    SELECT
      anuncios.*,
      users.nome AS nome_fornecedor
    FROM anuncios
    INNER JOIN users ON users.id = anuncios.fornecedor_id
    WHERE anuncios.status = 'ativo'
    ORDER BY anuncios.criado_em DESC
  `;

  db.all(sql, [], callback);
}
```

```
function buscarAnuncioPorId(id, callback) {
  const sql = `
    SELECT
      anuncios.*,
      users.nome AS nome_fornecedor,
      users.email AS email_fornecedor
    FROM anuncios
    INNER JOIN users ON users.id = anuncios.fornecedor_id
    WHERE anuncios.id = ?
  `;

  db.get(sql, [id], callback);
}
```

Validações Assíncronas: A lógica de negócio no *Controller* emprega buscas via *Promises* para validar dependências (como ID do fornecedor) antes de consolidar estruturas e enviá-las ao banco, prevenindo *rollbacks* complexos.

Trecho Real de Código: Validação com Promises

```
// 2. Buscar id_categoria pelo nome
const cat = await new Promise((resolve, reject) => {
  anuncioModel.buscarCategoriaPorNome(categoria, (err, row) => {
    if (err) reject(err);
    else resolve(row);
  });
});

if (!cat) {
  return res.status(400).json({ error: `Categoria "${categoria}" não encontrada` });
}
```

Algoritmos de Hashing (Segurança de Dados)

Além da estruturação de buscas, a aplicação emprega algoritmos de *Hashing Criptográfico* para garantir a segurança das credenciais na tabela `users`. A conversão de senhas em texto puro para hashes (utilizando bibliotecas como o `bcrypt`) aplica uma complexidade intencional e assimétrica de processamento. Ao invés de usar estruturas de busca rápida para senhas, o sistema exige um alto custo computacional no momento do cadastro e do login para calcular o *Salt* e a dispersão matemática, tornando a base de dados resiliente contra ataques de dicionário ou força bruta.

```
const bcrypt = require("bcrypt");

async function gerarHash(senha) {
  return await bcrypt.hash(senha, 10);
}

async function compararSenha(senha, hash) {
  return await bcrypt.compare(senha, hash);
}

module.exports = {
  gerarHash,
  compararSenha
};
```

3. Front-end: Manipulação de Dados na Tela

A interface do usuário processa os dados transitórios (JSON) enviados pelo Back-end, convertendo-os em elementos da DOM (Document Object Model) da forma mais eficiente possível.

- **Normalização de Dados *O(1)*:** Para tratar chaves inconsistentes vindas da API, foi implementado um algoritmo de busca linear condicional (função `campo`). Como o universo de chaves verificadas é estático e minúsculo, a complexidade opera muito próxima a ***O(1)***.
- **Iteração e Renderização *O(n)*:** Em vez de utilizar laços que forçam o redesenho da tela a cada iteração, o algoritmo aplica `.map()` seguido de `.join("")`. Isso percorre o Array em ***O(n)***, compila o HTML na memória e injeta na tela em uma única operação, anulando os gargalos de renderização do navegador.

Trecho Real de Código: Algoritmo de Renderização

```
    grid.innerHTML = anuncios.map((anuncio) => {
      const id = campo(anuncio, "id", "id_anuncio");
      const titulo = campo(anuncio, "titulo", "nome_produto") || "Produto sem título";
      const descricao = campo(anuncio, "descricao") || "Sem descrição cadastrada.";
      const categoria = campo(anuncio, "categoria", "nome_categoria") || "Produto";
      const fornecedor = campo(anuncio, "nome_fornecedor", "fornecedor") || "Fornecedor";

      return `
        <div class="product-card">
          <div class="product-img">
            <img src="${imagemDoAnuncio({ ...anuncio, id })}" alt="${titulo}" onerror="this.src='assets/images/eco-kit.png'">
          </div>
          <div class="product-info">
            <h4>${titulo}</h4>
            <p class="product-desc">${descricao}</p>
            <p class="product-desc"><strong>Categoria:</strong> ${categoria}</p>
            <p class="product-desc"><strong>Fornecedor:</strong> ${fornecedor}</p>
            <button class="product-btn" onclick="favoritarAnuncio(${id})">Favoritar</button>
          </div>
        </div>
      `;
    }).join("");
  } catch (error) {
    grid.innerHTML = `<p style="grid-column:1/-1;color:#9b1c1c;">Erro ao carregar anúncios: ${error.message}</p>`;
  }
}
```

4. Comparações de Desempenho (Benchmark Teórico)

A tabela de benchmark abaixo compara de forma analítica a abordagem ineficiente versus a arquitetura otimizada implementada na plataforma Kakuab Market. Ela ilustra como as decisões de estrutura de dados afetam diretamente o desempenho computacional.

Tarefa Analisada no Kakuab	Abordagem Ineficiente	Abordagem Otimizada Aplicada	Ganho de Complexidade
Encontrar um anúncio específico	Traz tudo para o Node.js e usa	Busca direto pelo ID com índice B-Tree (PK)	Cai de *O(n)* para
	`.find()`	-	*O(log n)*
Ordenar anúncios por data	`Array.sort()` no Servidor Node.js	`ORDER BY` processado no SQLite	Libera RAM e CPU do Servidor
Associar Fornecedor ao Anúncio	Laço de repetição no Back-end (N+1 queries)	Cláusula `INNER JOIN` indexada	Reduz drasticamente tráfego de rede e E/S
Renderizar 100 anúncios no Front-end	Laço `for` alterando o DOM 100 vezes	Algoritmo `.map().join("")`	Evita 99 *Reflows/ Repaints* no Navegador

5. Conclusão

A aplicação coerente de estruturas de dados transparece em todas as camadas da solução. A delegação de filtros, restrições e ordenações para as estruturas otimizadas em árvore do banco de dados relacional assegurou que o Back-end operasse de forma leve com complexidade logarítmica para acessos pontuais. A tabela de benchmark e a análise algorítmica comprovam que a arquitetura adotada atinge os requisitos de desempenho e escalabilidade da plataforma Kakuab Market, prevenindo gargalos clássicos de renderização e sobrecarga de memória.